

20. Faceted Filtering

Phase: L **Code:** `search_api/routers/category.py` , `search_api/routers/companies.py` , `search_api/models/requests.py` , `admin-ui/src/pages/SearchTest.jsx`

What it is

Faceted filtering is a search feature that lets users narrow results by selecting values from predefined categories (called "facets"). On an e-commerce site, when you search for "laptop" and then filter by Brand: Dell, Price: \$500-\$1000, RAM: 16GB -- those are facets.

In our system, the facets are metadata fields stored as JSONB in PostgreSQL. Each category or company row has a `metadata` column containing a JSON object with fields like `ai_type` (the classification level: Product, Category, Equipment, etc.) and `status` (OK or REVIEW). The user picks values from dropdowns in the Admin UI, and those selections become SQL WHERE clauses that constrain the search results.

The key architectural decision in our system is that faceted filtering is applied as **pre-filters** -- the WHERE clauses are added to both the vector search and the BM25 search queries before results are retrieved. This means the search only considers rows that match the selected filters, rather than searching everything and then removing non-matching results after the fact.

Real-life analogy

Imagine a library with 100,000 books. You want to find books about "machine learning."

Without faceted filtering: You ask the librarian for "machine learning" books. She searches the entire catalogue and returns 500 books. You then manually sort through all 500 to find the ones that are (a) in English and (b) published after 2020.

With pre-filtering (our approach): Before the librarian even starts searching, you say "Only look in the English section, and only books published after 2020." She goes to the English-post-2020 shelf (much smaller), searches for "machine

learning" there, and returns 50 highly relevant results. The search was faster because she searched a smaller set, and every result matches your constraints.

Post-filtering (the alternative): The librarian searches all 100,000 books for "machine learning", finds 500 results, then removes the ones that are not English or are pre-2020. This returns the same 50 results, but she did 10x more work, and her initial "top 50" results might have been dominated by non-English books, causing her to miss some good English ones that ranked 51st-100th.

Pre-filtering is better for our use case because it lets the vector search and BM25 search focus their ranking power on the subset that actually matters.

Why we used it here

R15 (Category search) and **R16 (Company matching)** both accept optional filter parameters. The client requirement is that users should be able to narrow search results by metadata fields like `ai_type` and `status` -- the classification taxonomy that the ETL pipeline generated.

We store metadata as PostgreSQL JSONB rather than separate columns because the metadata schema varies between categories and companies, and new fields may be added by the ETL enrichment step without requiring schema migrations. JSONB supports the `@>` (contains) operator, which lets us filter on any key-value pair without knowing the schema in advance.

The filters are applied as additional WHERE clauses in both the vector and BM25 SQL queries, making them true pre-filters that reduce the search space before ranking occurs.

Code walkthrough

1. Request model with optional filters --

`search_api/models/requests.py`

```
class CategorySearchRequest(BaseModel):
    query: str = Field(..., min_length=1, description="Free-text search query")
    limit: int = Field(10, ge=1, le=100)
    offset: int = Field(0, ge=0, description="Pagination offset")
    language: str | None = Field(None, description="Force language; auto-detected if c
    # The filters field is an optional dictionary of key-value pairs
    # Example: {"ai_type": "Product"} or {"status": "OK"}
    # None means "no filtering" -- return all matching results
```

```
filters: dict | None = Field(None, description="JSONB metadata pre-filters")
rerank: bool = Field(False, description="Re-rank using cosine similarity blending")
```

The `filters` field is intentionally typed as `dict | None` rather than a rigid Pydantic model with specific fields. This allows the API to accept any metadata key-value pair as a filter, without needing to update the request schema every time a new metadata field is added by the ETL pipeline.

2. Building SQL WHERE clauses from filters -- `search_api/routers/category.py`

```
def _build_jsonb_filter(filters: dict | None) -> tuple[str, tuple]:
    # No filters? Return empty additions to the SQL
    if not filters:
        return "", ()

    clauses = []
    params = []
    for key, value in filters.items():
        # Each filter becomes: AND metadata @> '{"ai_type": "Product"}'::jsonb
        # The @> operator means "contains" -- it checks if the metadata JSONB
        # object contains the specified key-value pair
        clauses.append("AND metadata @> %s::jsonb")
        params.append(json.dumps({key: value}))
    return " ".join(clauses), tuple(params)
```

Let's trace through an example:

- Input: `filters = {"ai_type": "Product", "status": "OK"}`
- Iteration 1: `clauses = ["AND metadata @> %s::jsonb"]`, `params = ['{"ai_type": "Product"}']`
- Iteration 2: `clauses = ["AND metadata @> %s::jsonb", "AND metadata @> %s::jsonb"]`, `params = ['{"ai_type": "Product"}', '{"status": "OK"}']`
- Return: `("AND metadata @> %s::jsonb AND metadata @> %s::jsonb", ('{"ai_type": "Product"}', '{"status": "OK"}'))`

This produces parameterized SQL (using `%s` placeholders instead of string interpolation), which prevents SQL injection. The JSON is serialized by Python's `json.dumps()` and passed to PostgreSQL as a parameter, never concatenated into the SQL string.

3. How filters integrate with vector and BM25 search -- `search_api/services/search.py`

```
def _vector_search(cur, table, query_vec, lang, limit, id_col,
                  extra_where="", extra_params=()):
    sql = f"""
        SELECT {id_col}, language,
               1 - (embedding <=> %s::vector) AS cosine_similarity
        FROM {table}
        WHERE language = %s {extra_where}
        --             ^^^^^^^^^^^ filters injected here
        ORDER BY embedding <=> %s::vector
        LIMIT %s
    """
    cur.execute(sql, (vec_literal, lang) + extra_params + (vec_literal, limit))
    #             ^^^^^^^^^^^ filter params injected here
    return cur.fetchall()
```

The `extra_where` string (e.g., `"AND metadata @> %s::jsonb"`) is appended directly to the WHERE clause, and the corresponding `extra_params` tuple is spliced into the parameter list. This means:

- The vector search only computes cosine distance for rows that match the filters.
- The BM25 search only checks tsvector matches for rows that match the filters.
- The LIMIT applies after filtering, so you always get the requested number of results from the filtered subset.

4. The router wires it together -- `search_api/routers/category.py`

```
@router.post("/search/category", response_model=CategorySearchResponse)
def search_category(req: CategorySearchRequest) -> CategorySearchResponse:
    # Step 1: Convert the request's filters dict into SQL clause + params
    extra_where, extra_params = _build_jsonb_filter(req.filters)

    # Step 2: Pass them into search_table, which passes them to both
    # _vector_search and _bm25_search
    raw = search_table(
        table="category_vectors",
        query=req.query,
        limit=req.limit,
        offset=req.offset,
        lang_override=req.language,
        extra_where=extra_where,      # ← the AND clauses
        extra_params=extra_params,    # ← the JSON values
```

```
    rerank=req.rerank,  
  )
```

5. The Admin UI filter controls -- `admin-ui/src/pages/SearchTest.jsx`

```
// State for the two filter dropdowns  
const [filterAiType, setFilterAiType] = useState("");  
const [filterStatus, setFilterStatus] = useState("");  
  
// In the search handler, build the filters object from dropdown values  
const filters = {};  
if (filterAiType) filters.ai_type = filterAiType;    // e.g. "Product"  
if (filterStatus) filters.status = filterStatus;    // e.g. "OK"  
const hasFilters = Object.keys(filters).length > 0 ? filters : undefined;  
  
// Pass to the API call  
data = await searchCategories(query, language, limit, hasFilters, offset, rerank);
```

The UI provides hardcoded dropdown options for `ai_type` (Product, Category, Subcategory, Equipment, Service, System, Technology, Solution, Component, Level 3, Level 4, Level 5) and `status` (OK, REVIEW). When the user selects a value, it is included in the filters dict sent to the API; when "All types" / "All statuses" is selected (empty string), the key is omitted.

6. The PostgreSQL `@>` (contains) operator

The `@>` operator is PostgreSQL's JSONB containment check. It returns true if the left-hand JSONB value contains the right-hand JSONB value:

```
-- True: the metadata object contains {"ai_type": "Product"}  
'{"ai_type": "Product", "status": "OK", "level": 3}':::jsonb  
  @> '{"ai_type": "Product"}':::jsonb  
-- → true  
  
-- False: the metadata does not contain {"ai_type": "Service"}  
'{"ai_type": "Product", "status": "OK", "level": 3}':::jsonb  
  @> '{"ai_type": "Service"}':::jsonb  
-- → false
```

PostgreSQL can use a GIN index on the `metadata` column to speed up `@>` queries, making JSONB filtering efficient even on large tables.

How to explain this in an interview

"We implement faceted filtering using PostgreSQL's JSONB containment operator (`@>`). Each search result has a `metadata` JSONB column with fields like `ai_type` and `status` . When the user selects filter values in the UI, we dynamically build parameterized SQL WHERE clauses -- `AND metadata @> '{"ai_type": "Product"}'::jsonb` -- and inject them into both the vector search and BM25 search queries. This makes them true pre-filters: the search engine only ranks rows that match the filter criteria, rather than searching everything and discarding non-matching results afterwards. We use JSONB rather than fixed columns because the metadata schema is flexible and driven by the ETL enrichment pipeline, and the `@>` operator can be GIN-indexed for performance."